

CampusRide - Design Document

1. Problem Understanding

Large educational campuses often face transportation challenges such as long waiting times, inefficient vehicle utilization, lack of ride visibility, and communication gaps between riders and drivers. CampusRide addresses these issues through a centralized platform that enables users to request rides, track ride status, and manage transportation operations efficiently.

Objectives:

- Reduce ride request processing time.
- Improve campus transportation efficiency.
- Provide real-time ride visibility.
- Ensure a secure and scalable solution.
- Improve user experience for students, faculty, and drivers.

2. System Architecture

CampusRide follows a three-tier architecture:

Presentation Layer:

- React-based frontend.
- Responsive dashboards for riders and drivers.
- Real-time ride status updates.

Application Layer:

- Node.js and Express backend.
- Business logic for authentication, ride allocation, notifications, and ride lifecycle management.
- REST API communication.

Data Layer:

- MongoDB database.
- Stores user profiles, rides, ratings, and notifications.

Workflow:

1. User submits a ride request.
2. Backend validates request.
3. Driver receives request.
4. Driver accepts/rejects.
5. Ride progresses through status updates.
6. Completion data is stored for analytics.

3. Functional Requirements

- User registration and login.
- Ride booking and cancellation.
- Driver ride acceptance and completion.
- Ride status tracking.
- Rating and feedback management.
- Notification delivery.
- Admin monitoring and reporting.

4. Non-Functional Requirements

- Scalability for growing user populations.
- Security using JWT authentication.
- High availability.
- Responsive user interface.
- Reliable database transactions.
- Maintainable and modular codebase.

5. Database Schema

Users:

userId, name, email, passwordHash, role, createdAt

Drivers:

driverId, userId, status, vehicleDetails, rating

Rides:

rideId, riderId, driverId, pickupLocation, destination, status, requestTime, completionTime

Ratings:

ratingId, rideId, userId, score, feedback

Notifications:

notificationId, userId, message, timestamp, readStatus

Relationships:

- One user can create multiple rides.
- One driver can complete multiple rides.
- One ride can receive ratings.
- Notifications are associated with users.

6. Entity Relationship Diagram (ERD)

USER (1) ---- (M) RIDE
 DRIVER (1) ---- (M) RIDE
 RIDE (1) ---- (M) RATING
 USER (1) ---- (M) NOTIFICATION

Explanation:

The ERD models the relationship between users, drivers, ride transactions, and feedback. Each ride links riders and drivers while maintaining operational history.

7. API Overview

Authentication APIs:

POST /api/auth/register

POST /api/auth/login

Ride APIs:

GET /api/rides

POST /api/rides/request

PUT /api/rides/:id/accept

PUT /api/rides/:id/reject

PUT /api/rides/:id/complete

Driver APIs:

GET /api/drivers/dashboard

GET /api/drivers/rides

Rating APIs:
POST /api/ratings
GET /api/ratings/:rideId

Notification APIs:
GET /api/notifications
PUT /api/notifications/read

8. Design Decisions

Frontend:
React was selected for component reusability and responsive design.

Backend:
Node.js and Express provide asynchronous processing suitable for real-time ride operations.

Database:
MongoDB supports flexible schemas and rapid development.

Security:
JWT authentication protects API endpoints.
Passwords are stored using hashing techniques.

Deployment:
The architecture supports deployment on cloud platforms and horizontal scaling.

9. Security Considerations

- JWT-based authentication.
- Password hashing.
- Input validation.
- API authorization checks.
- Secure environment variable management.
- Protection against unauthorized access.

10. Future Enhancements

- GPS-based live tracking.
- Ride sharing optimization.
- AI-powered ride allocation.
- Mobile application support.
- Analytics dashboard.
- Emergency assistance features.

11. Conclusion

CampusRide provides a modern ride management platform tailored for campus transportation. The solution combines a responsive frontend, scalable backend services, secure authentication mechanisms, and efficient ride allocation workflows. The modular architecture enables future expansion while maintaining usability and performance.